

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Desarrollo Ejercicios _Semana 12

CURSO:

- Base de Datos II

DOCENTE:

- Fernández Bejarano Raúl Enrique

ESTUDIANTE:

- Rosales Crispín Aristóteles Onassis

Ciclo:

- 5°SEMESTRE

HUANCAYO- JUNIN
2026

TEMA 1: ESTRATEGIAS DE BACKUP: COMPLETO, DIFERENCIAL Y TRANSACCIONAL

Proyecto 1: Respaldo Completo Institucional Diario

Enunciado:

Crear un procedimiento que genere un respaldo completo de la base de datos y registre la fecha de ejecución.

```
USE master;
GO

-- 1. Crear un procedimiento almacenado para respaldo completo con registro
CREATE OR ALTER PROCEDURE dbo.SP_BackupCompleto_GradosTitulos
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @backupPath NVARCHAR(500);
    DECLARE @fileName NVARCHAR(500);
    DECLARE @timestamp NVARCHAR(20);

    -- Obtener fecha y hora actual para el nombre del archivo
    SET @timestamp = FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss');
    SET @backupPath = 'C:\Backups\GradosTitulos\Completo\';
    SET @fileName = @backupPath + 'GradosTitulos_Full_' + @timestamp + '.bak';

    -- Realizar el respaldo completo
    BACKUP DATABASE GradosTitulos
    TO DISK = @fileName
    WITH
        FORMAT, -- Sobrescribe si existe
        INIT, -- Inicializa el conjunto de medios
        NAME = 'Backup completo de GradosTitulos - ' + @timestamp,
        COMPRESSION, -- Recomendado para reducir espacio
        STATS = 10; -- Muestra progreso cada 10%

    -- Registrar en una tabla de log (opcional)
    IF OBJECT_ID('dbo.BackupLog') IS NULL
    BEGIN
        CREATE TABLE dbo.BackupLog (
            Id INT IDENTITY(1,1) PRIMARY KEY,
            TipoBackup VARCHAR(20),
            FechaInicio DATETIME2,
            FechaFin DATETIME2,
            Archivo NVARCHAR(500),
            TamanoMB DECIMAL(18,2),
            Estado VARCHAR(20)
        );
    END

    INSERT INTO dbo.BackupLog (TipoBackup, FechaInicio, FechaFin, Archivo,
Estado)
VALUES (
    'COMPLETO',
    GETDATE(),
    GETDATE(),
    @fileName,
    'EXITOSO'
);

-- Nota: En producción se captura el tamaño real desde los archivos.
```

```

        PRINT 'Backup completo completado: ' + @fileName;
END;
GO

-- 2. Programar el procedimiento con SQL Server Agent (paso opcional)
-- Se puede crear un trabajo que ejecute: EXEC
dbo.SP_BackupCompleto_GradosTitulos;
-- con frecuencia diaria (ej. a las 2:00 AM)

```

Justificación

Protege contra fallos de disco o corrupción. Es la base de cualquier estrategia de recuperación.

Buenas prácticas

- Usar compresión para ahorrar espacio.
- Almacenar en una ubicación distinta al disco de datos.
- Verificar la integridad con RESTORE VERIFYONLY.

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos Enunciado:

Diseña una estrategia que combine respaldos completos semanales y diferenciales diarios.

```

USE master;
GO

-- 1. Procedimiento para backup completo semanal (ejecutar los domingos)
CREATE OR ALTER PROCEDURE dbo.SP_BackupCompletoSemanal_Grados
AS
BEGIN
    DECLARE @path NVARCHAR(500) = 'C:\Backups\GradosTitulos\Completo\';
    DECLARE @file NVARCHAR(500) = @path + 'GradosTitulos_Full_' +
        FORMAT(GETDATE(), 'yyyyMMdd') + '.bak';
    BACKUP DATABASE GradosTitulos
    TO DISK = @file
    WITH COMPRESSION, STATS = 10;
    -- Registrar en log
    INSERT INTO dbo.BackupLog (TipoBackup, FechaInicio, FechaFin, Archivo,
Estado)
    VALUES ('COMPLETO_SEMANAL', GETDATE(), GETDATE(), @file, 'EXITOSO');
END;
GO

-- 2. Procedimiento para backup diferencial diario (ejecutar de lunes a sábado)
CREATE OR ALTER PROCEDURE dbo.SP_BackupDiferencialDiario_Grados
AS
BEGIN
    DECLARE @path NVARCHAR(500) = 'C:\Backups\GradosTitulos\Diferencial\';
    DECLARE @file NVARCHAR(500) = @path + 'GradosTitulos_Diff_' +
        FORMAT(GETDATE(), 'yyyyMMdd') + '.diff';
    BACKUP DATABASE GradosTitulos
    TO DISK = @file
    WITH DIFFERENTIAL, COMPRESSION, STATS = 10;
    INSERT INTO dbo.BackupLog (TipoBackup, FechaInicio, FechaFin, Archivo,
Estado)
    VALUES ('DIFERENCIAL', GETDATE(), GETDATE(), @file, 'EXITOSO');
END;

```

GO

```
-- 3. Programar tareas:  
-- - Ejecutar SP_BackupCompletoSemanal_Grados cada domingo a las 2:00 AM.  
-- - Ejecutar SP_BackupDiferencialDiario_Grados de lunes a sábado a las 2:00 AM.  
-- (Ajustar según necesidades)
```

Justificación

Reduce el tiempo de backup respecto al completo y acelera la restauración al necesitar solo el último full y el último diff.

Buenas prácticas

- El diferencial debe basarse en el último full.
- No usar diff sin full previo.
- Monitorear el tamaño del diff, ya que crece a lo largo de la semana.

Proyecto 3: Estrategia Integral Full + Differential + Log Backup

Enunciado:

Diseñar estrategia con full semanal, diferencial diario y log cada 30 minutos (24/7).

USE master;

GO

```
-- 1. Asegurar que el modelo de recuperación esté en FULL (necesario para log backups)
```

```
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
```

GO

```
-- 2. Procedimiento para backup del log de transacciones (cada 30 min)
```

```
CREATE OR ALTER PROCEDURE dbo.SP_BackupLog_Grados
```

```
AS
```

```
BEGIN
```

```
    DECLARE @path NVARCHAR(500) = 'C:\Backups\GradosTitulos\Logs\';
```

```
    DECLARE @file NVARCHAR(500) = @path + 'GradosTitulos_Log_' +
```

```
FORMAT(GETDATE(), 'yyyymmdd_HHmmss') + '.trn';
```

```
    BACKUP LOG GradosTitulos
```

```
    TO DISK = @file
```

```
    WITH COMPRESSION;
```

```
    INSERT INTO dbo.BackupLog (TipoBackup, FechaInicio, FechaFin, Archivo,
```

```
Estado)
```

```
    VALUES ('LOG', GETDATE(), GETDATE(), @file, 'EXITOSO');
```

```
END;
```

GO

```
-- 3. Procedimientos para full semanal y diferencial diario (similares a Proyecto 2)
```

```
-- Se pueden reutilizar o crear versiones ajustadas.
```

```
-- 4. Programación de trabajos:
```

```
-- - Completo: domingo 2:00 AM
```

```
-- - Diferencial: lunes a sábado 2:00 AM
```

```
-- - Log: cada 30 minutos (00:00, 00:30, ... 23:30)
```

Justificación

Ofrece recuperación punto a punto (point-in-time). Indispensable para bases con alta actividad y requisitos de RPO ajustados.

Buenas prácticas

- Realizar pruebas de restauración periódicas (ej. mensuales) para medir RTO real.
- Mantener backups en múltiples ubicaciones geográficas.
- Documentar el procedimiento de recuperación y automatizar lo máximo posibles
- Considerar usar RESTORE WITH NORECOVERY para mantener la base en estado de restauración y aplicar logs posteriores.

Proyecto 4: Diseño de Estrategia de Respaldo para Alta Disponibilidad (RTO 15 min)

Enunciado:

Definir respaldos completos, diferenciales y transaccionales para garantizar recuperación máxima de 15 minutos.

```
-- Estrategia para RTO < 15 minutos:
-- Se debe combinar:
-- - Full semanal (domingo)
-- - Diferencial diario (lunes a sábado)
-- - Log cada 5 minutos (para lograr recuperación en menos de 15 min)
-- Además, se recomienda utilizar mirroring, Always On Availability Groups o
Log Shipping.
-- Pero en términos de backup, se reduce el intervalo del log.

-- Modificar el procedimiento de log para ejecutarse cada 5 minutos:
CREATE OR ALTER PROCEDURE dbo.SP_BackupLog_Grados_Rapido
AS
BEGIN
    DECLARE @path NVARCHAR(500) = 'C:\Backups\GradosTitulos\Logs\';
    DECLARE @file NVARCHAR(500) = @path + 'GradosTitulos_Log_' +
FORMAT(GETDATE(), 'yyyyMMdd_HHmss') + '.trn';
    BACKUP LOG GradosTitulos
    TO DISK = @file
    WITH COMPRESSION, INIT;
    INSERT INTO dbo.BackupLog (TipoBackup, FechaInicio, FechaFin, Archivo,
Estado)
    VALUES ('LOG_5MIN', GETDATE(), GETDATE(), @file, 'EXITOSO');
END;
GO

-- Programar cada 5 minutos (o incluso 10 minutos) para cumplir RTO.
-- También se recomienda tener una segunda copia en otra ubicación
(redundancia).
-- Además, practicar la restauración regul
```

Justificación

Para lograr recuperación rápida se necesita: backups de log muy frecuentes, redundancia de copias y, preferiblemente, usar tecnologías como Always On o Log Shipping para reducir aún más el tiempo de restauración.

Buenas prácticas

- Realizar pruebas de restauración periódicas (ej. mensuales) para medir RTO real.
- Mantener backups en múltiples ubicaciones geográficas
- Documentar el procedimiento de recuperación y automatizar lo máximo posible.
- Considerar usar RESTORE WITH NORECOVERY para mantener la base en estado de restauración y aplicar logs posteriores.

Proyecto 5: Estrategia Corporativa de Respaldo para GradosTítulos

Enunciado:

La Universidad ha identificado que la base de datos GradosTítulos es crítica. Se requiere implementar una estrategia avanzada que garantice una pérdida máxima de información de 15 minutos. Diseñe una solución que incluya:

- Respaldo completo cada domingo.
- Respaldo diferencial diario.
- Respaldo del log de transacciones cada 15 minutos
- Compresión de respaldos.
- Verificación automática de integridad.

```
-- =====
-- PROYECTO 5: Estrategia Corporativa de Respaldo
-- =====
-- Se asume que la base de datos está en modo de recuperación FULL.
-- Verificar y cambiar si es necesario:
ALTER DATABASE GradosTitulos SET RECOVERY FULL;
GO

-- 1. Crear un procedimiento para el respaldo completo (ejecutar cada domingo)
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_FullBackup
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @backupPath NVARCHAR(500) = 'C:\Backups\GradosTitulos\Full\';
    DECLARE @fileName NVARCHAR(500) = @backupPath + 'GradosTitulos_Full_'
        + FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + '.bak';

    -- Crear la carpeta si no existe (ejecutar xp_create_subdir)
    EXEC xp_create_subdir @backupPath;

    BACKUP DATABASE GradosTitulos
    TO DISK = @fileName
    WITH
        COMPRESSION,
        INIT,
        CHECKSUM,
        STATS = 10;

    -- Verificar la integridad del respaldo
    RESTORE VERIFYONLY FROM DISK = @fileName;
END;
GO

-- 2. Procedimiento para respaldo diferencial (ejecutar cada día, excepto
-- domingo)
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_DiffBackup
AS
BEGIN
```

```

SET NOCOUNT ON;
DECLARE @backupPath NVARCHAR(500) = 'C:\Backups\GradosTitulos\Diff\';
DECLARE @fileName NVARCHAR(500) = @backupPath + 'GradosTitulos_Diff_'
    + FORMAT(GETDATE(), 'yyyyMMdd') + '.bak';

EXEC xp_create_subdir @backupPath;

BACKUP DATABASE GradosTitulos
TO DISK = @fileName
WITH
    DIFFERENTIAL,
    COMPRESSION,
    INIT,
    CHECKSUM,
    STATS = 10;

RESTORE VERIFYONLY FROM DISK = @fileName;
END;
GO

-- 3. Procedimiento para respaldo del log de transacciones (cada 15 minutos)
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_LogBackup
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @backupPath NVARCHAR(500) = 'C:\Backups\GradosTitulos\Log\';
    DECLARE @fileName NVARCHAR(500) = @backupPath + 'GradosTitulos_Log_'
        + FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + '.trn';

    EXEC xp_create_subdir @backupPath;

    BACKUP LOG GradosTitulos
    TO DISK = @fileName
    WITH
        COMPRESSION,
        INIT,
        CHECKSUM,
        STATS = 5;

    RESTORE VERIFYONLY FROM DISK = @fileName;
END;
GO

-- 4. (Opcional) Crear un trabajo de SQL Agent para automatizar:
--     - Full: todos los domingos a las 02:00
--     - Diff: todos los días (excepto domingo) a las 02:00
--     - Log: cada 15 minutos, de 06:00 a 23:59

-- Ejemplo de creación de job (comentado, se puede ejecutar manualmente):
/*
USE msdb;
GO

EXEC dbo.sp_add_job
    @job_name = N'Backup_GradosTitulos_Full',
    @enabled = 1,
    @description = N'Backup completo semanal';
EXEC dbo.sp_add_jobstep
    @job_name = N'Backup_GradosTitulos_Full',
    @step_name = N'Full backup',
    @command = N'EXEC Backup.GradosTitulos_FullBackup;';
EXEC dbo.sp_add_schedule

```

```

        @schedule_name = N'Schedule_Full_Sunday_2am',
        @freq_type = 8,                -- semanal
        @freq_interval = 1,           -- domingo
        @active_start_time = 020000;
EXEC dbo.sp_attach_schedule
        @job_name = N'Backup_GradosTitulos_Full',
        @schedule_name = N'Schedule_Full_Sunday_2am';
EXEC dbo.sp_add_jobserver
        @job_name = N'Backup_GradosTitulos_Full';
*/

```

Justificación técnica

La combinación de Full semanal + Diff diario + Log cada 15 minutos permite un RPO de 15 minutos (pérdida máxima de datos). Con compresión se reduce espacio y tiempo de transferencia. La verificación con RESTORE VERIFYONLY garantiza la integridad de los backups antes de confiar en ellos.

Buenas prácticas

- Mantener la base de datos en modo FULL RECOVERY.
- Programar los backups en horas de baja actividad.
- Monitorear el espacio en disco y retener respaldos según política (ej. 7 días).
- Probar periódicamente las restauraciones en un entorno de prueba.
- Utilizar CHECKSUM para detectar corrupción.

Proyecto 6: Diseño de Estrategia de Respaldo para Recuperación ante Desastres

Enunciado:

La Oficina de TI requiere implementar una estrategia de Disaster Recovery para la base de datos GradosTítulos. Diseñe un esquema de respaldo que permita reconstruir completamente la base de datos en un servidor alternativo ante la pérdida total del servidor principal.

```

-- =====
-- PROYECTO 6: Estrategia de Recuperación ante Desastres (DR)
-- =====
-- Concepto: Se necesita tener respaldos completos (full) que se copien a un
servidor remoto,
-- y un plan de restauración documentado.

-- 1. Procedimiento de respaldo completo con copia a ubicación secundaria
-- Se usa COPY_ONLY para no interrumpir la cadena de logs si ya hay
respaldos programados.
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_DR_FullBackup
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @localPath NVARCHAR(500) = 'C:\Backups_DR\GradosTitulos\Full\';
    DECLARE @remotePath NVARCHAR(500) =
'\\ServidorRemoto\Backups\GradosTitulos\Full\';
    DECLARE @fileName NVARCHAR(500) = 'GradosTitulos_Full_DR_'
+ FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss') + '.bak';
    DECLARE @localFile NVARCHAR(500) = @localPath + @fileName;
    DECLARE @remoteFile NVARCHAR(500) = @remotePath + @fileName;

    -- Crear directorios locales y remotos (asumiendo que el remoto está
mapeado)
    EXEC xp_create_subdir @localPath;
    -- Para el remoto, se podría usar xp_cmdshell, pero se recomienda usar un
script externo.

```



```

-- Aquí solo se genera el respaldo local.

BACKUP DATABASE GradosTitulos
TO DISK = @localFile
WITH
    COPY_ONLY,          -- No afecta la cadena de respaldos diferenciales
    COMPRESSION,
    INIT,
    CHECKSUM,
    STATS = 10;

-- Verificar integridad
RESTORE VERIFYONLY FROM DISK = @localFile;

-- Copiar el archivo al servidor remoto (usando xp_cmdshell o un job
externo)
-- Ejemplo con xp_cmdshell (requiere habilitar)
-- DECLARE @cmd NVARCHAR(4000) = 'copy "' + @localFile + '" "' +
@remoteFile + '"';
-- EXEC xp_cmdshell @cmd;
END;
GO

-- 2. Respaldo de la base de datos master y msdb (necesarias para la
restauración en el servidor alternativo)
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_DR_SystemBackup
AS
BEGIN
    -- Backup de master
    BACKUP DATABASE master
    TO DISK = 'C:\Backups_DR\System\master_' + FORMAT(GETDATE(), 'yyyyMMdd') +
'.bak'
    WITH COPY_ONLY, COMPRESSION, CHECKSUM;

    -- Backup de msdb
    BACKUP DATABASE msdb
    TO DISK = 'C:\Backups_DR\System\msdb_' + FORMAT(GETDATE(), 'yyyyMMdd') +
'.bak'
    WITH COPY_ONLY, COMPRESSION, CHECKSUM;
END;
GO

-- 3. Procedimiento de restauración en servidor alternativo (ejemplo)
-- Este procedimiento se ejecutaría en el servidor de respaldo, usando los
archivos copiados.
CREATE OR ALTER PROCEDURE Backup.GradosTitulos_DR_Restore
@backupFile NVARCHAR(500)
AS
BEGIN
    -- Restaurar la base de datos con NORECOVERY para luego aplicar logs y
diferenciales
    RESTORE DATABASE GradosTitulos
    FROM DISK = @backupFile
    WITH
        MOVE 'GradosTitulos' TO 'D:\Data\GradosTitulos.mdf',
        MOVE 'GradosTitulos_log' TO 'D:\Logs\GradosTitulos_log.ldf',
        NORECOVERY,
        REPLACE;

    -- Aplicar diferenciales y logs según sea necesario (orden secuencial)
    -- Ejemplo: RESTORE DATABASE GradosTitulos FROM DISK = ... WITH NORECOVERY;
    -- Finalmente: RESTORE DATABASE GradosTitulos WITH RECOVERY;

```

```
END;  
GO
```

```
-- 4. Recomendación: Establecer un job que ejecute el respaldo DR completo  
diariamente  
-- y que copie los archivos a una ubicación externa (Azure Blob, AWS S3, o  
servidor remoto).
```

Justificación técnica

La copia de respaldos a un servidor alternativo (físicamente separado) protege contra desastres (incendio, fallo de hardware, etc.). Se usan COPY_ONLY para no romper la cadena de respaldos diferenciales/logs existentes. Incluir respaldos de master y msdb es esencial para reconstruir el servidor completo.

Buenas prácticas

- Automatizar la copia de archivos mediante scripts (Robocopy, rsync, Azure CLI).
- Asegurar que el servidor alternativo tenga la misma versión de SQL Server.
- Documentar paso a paso el proceso de restauración y practicar simulacros.
- Mantener las claves de cifrado (si se usa TDE) en una bóveda accesible para el DR.
- Implementar alertas para fallos de respaldo o copia.

TEMA 2. RESTAURACION DE BASES DE DATOS EN DISTINTOS ESCENARIOS

Proyecto 1: Recuperación por eliminación accidental de registros

Enunciado:

Un administrador eliminó accidentalmente registros relacionados con expedientes de titulación. Restaure la base de datos utilizando el respaldo más reciente.

```
-- Supongamos que se desea restaurar la base de datos completa a un momento anterior a la eliminación.
-- Se necesita el último respaldo completo y los logs hasta justo antes del borrado.
-- Se asume que se conoce la hora del borrado (ej. '2026-06-28 10:30:00').

-- 1. Restaurar el último respaldo completo con NORECOVERY
RESTORE DATABASE GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak'
WITH NORECOVERY, REPLACE;

-- 2. Restaurar el último respaldo diferencial (si existe) con NORECOVERY
-- (puede ser el más reciente anterior a la hora de borrado)
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos\Diff\GradosTitulos_Diff_20260628.bak'
WITH NORECOVERY;

-- 3. Restaurar los logs de transacciones secuencialmente hasta el momento justo anterior al borrado.
-- Se debe restaurar cada archivo de log en orden.
RESTORE LOG GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_100000.trn'
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_101500.trn'
WITH NORECOVERY;

-- Supongamos que el borrado ocurrió a las 10:30, entonces restauramos hasta el log de las 10:15 y luego
-- no se aplican logs posteriores. Finalmente, recuperamos la base.
RESTORE DATABASE GradosTitulos WITH RECOVERY;

-- Opcional: Para recuperar hasta un punto específico en el tiempo (STOPAT)
-- En lugar de restaurar logs uno a uno, se puede usar STOPAT en el último log:
RESTORE LOG GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_101500.trn'
WITH STOPAT = '2026-06-28T10:29:59', RECOVERY;
```

Justificación técnica

Se restaura la base a un punto anterior al incidente usando la cadena de respaldos. El uso de STOPAT permite detener la restauración justo antes del evento no deseado.

Buenas prácticas

- Tener siempre disponibles los respaldos de logs para puntos en el tiempo.
- Documentar la hora exacta del incidente.
- Practicar restauraciones con STOPAT.

Proyecto 2: Restauración en un Servidor Alternativo

Enunciado:

La universidad requiere habilitar un servidor de contingencia para pruebas y recuperación. Restaure la base de datos GradosTítulos en una nueva instancia SQL Server.

```
-- En el servidor alternativo, se copian los archivos de respaldo.
-- Se restauran los archivos con diferentes rutas físicas.

-- 1. Restaurar el respaldo completo con MOVE para cambiar ubicación de
archivos.
RESTORE DATABASE GradosTítulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTítulos\Full\GradosTítulos_Full_20260627.bak
'
WITH
    MOVE 'GradosTítulos' TO 'D:\Data\GradosTítulos.mdf',
    MOVE 'GradosTítulos_log' TO 'E:\Logs\GradosTítulos.ldf',
    NORECOVERY,
    REPLACE;

-- 2. Aplicar el último diferencial (si se tiene)
RESTORE DATABASE GradosTítulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTítulos\Diff\GradosTítulos_Diff_20260628.bak
'
WITH NORECOVERY;

-- 3. Aplicar los logs de transacciones (opcional, hasta el momento más
reciente)
RESTORE LOG GradosTítulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTítulos\Log\GradosTítulos_Log_20260628_10150
0.trn'
WITH NORECOVERY;

-- ... (restaurar todos los logs necesarios)

-- 4. Dejar la base en estado operativo
RESTORE DATABASE GradosTítulos WITH RECOVERY;

-- Si se desea restaurar sin logs (solo full), se puede usar WITH RECOVERY
directamente.
```

Justificación técnica

Permite poner en marcha un entorno de respaldo o pruebas. Se utilizan MOVE para reubicar archivos físicos según la estructura del nuevo servidor.

Buenas prácticas

- Verificar que la versión de SQL Server sea compatible.
- Asegurar que las rutas de destino existan y tengan espacio suficiente.
- Probar la restauración periódicamente.

Proyecto 3: Recuperación después de Corrupción del Archivo MDF

Enunciado:

El archivo principal de datos presenta corrupción física. Implemente el procedimiento de restauración completo utilizando respaldos disponibles.

```
-- 1. Verificar la integridad de la base (opcional, pero se sabe que está corrupta).
-- Si la base está dañada, puede que no se pueda hacer un backup de log, pero se tiene la cadena de respaldos.

-- 2. Restaurar el último respaldo completo válido (con CHECKSUM y VERIFYONLY).
RESTORE VERIFYONLY FROM DISK =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627.bak';
-- Si pasa la verificación, se procede.

RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627.bak'
WITH NORECOVERY, REPLACE;

-- 3. Aplicar el último diferencial (si existe)
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos\Diff\GradosTitulos_Diff_20260628.bak'
WITH NORECOVERY;

-- 4. Aplicar todos los logs de transacciones desde el último diferencial hasta el momento actual.
-- Se deben restaurar en orden cronológico.
RESTORE LOG GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_100000.trn'
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK =
'C:\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_101500.trn'
WITH NORECOVERY;

-- ... (todos los logs hasta el último disponible)

-- 5. Recuperar la base
RESTORE DATABASE GradosTitulos WITH RECOVERY;
```

Justificación

Cuando el archivo MDF está corrupto, se debe restaurar desde los respaldos, ya que no se puede reparar in situ. Se aplican full, diff y logs para obtener la información más reciente posible.

Buenas prácticas

- Utilizar RESTORE VERIFYONLY antes de restaurar.
- Mantener respaldos regulares y verificar su integridad.
- Considerar el uso de WITH CHECKSUM al hacer respaldos.

Proyecto 4: Recuperación ante Desastre Total del Servidor

Enunciado:

Una falla de almacenamiento destruyó completamente el servidor donde se encontraba la base GradosTítulos. Diseñe el proceso de recuperación completa.

```
-- Este proceso se realiza en un nuevo servidor (o en el mismo luego de reparar hardware).
-- Se necesitan: respaldo completo del sistema (master, msdb) y los respaldos de la base de datos.

-- 1. Instalar SQL Server (misma versión y configuración).
-- 2. Restaurar las bases de sistema (master y msdb) para recuperar configuraciones y trabajos.
RESTORE DATABASE master FROM DISK = 'C:\Backups_DR\System\master_20260627.bak'
WITH REPLACE;
-- (Requiere iniciar SQL en modo single-user)

RESTORE DATABASE msdb FROM DISK = 'C:\Backups_DR\System\msdb_20260627.bak' WITH
REPLACE;

-- 3. Restaurar la base de datos GradosTitulos siguiendo el mismo procedimiento que en el proyecto 3,
-- pero utilizando los respaldos almacenados en una ubicación segura (remota o en cinta).
RESTORE DATABASE GradosTitulos
FROM DISK = 'Z:\Backups_DR\GradosTitulos\Full\GradosTitulos_Full_20260627.bak'
WITH NORECOVERY, REPLACE;

RESTORE DATABASE GradosTitulos
FROM DISK = 'Z:\Backups_DR\GradosTitulos\Diff\GradosTitulos_Diff_20260628.bak'
WITH NORECOVERY;

-- Aplicar todos los logs hasta el último disponible.
RESTORE LOG GradosTitulos FROM DISK = ... WITH NORECOVERY;
-- ...

RESTORE DATABASE GradosTitulos WITH RECOVERY;

-- 4. Verificar la integridad de la base restaurada.
DBCC CHECKDB(GradosTitulos);
```

Justificación técnica

Se restaura todo el servidor (sistema y base de datos) desde respaldos almacenados fuera del sitio. Es la estrategia más completa y requiere planificación y pruebas.

Buenas prácticas

- Almacenar respaldos en ubicaciones remotas (off-site).
- Documentar el procedimiento paso a paso.
- Realizar simulacros de desastre periódicos. • Incluir respaldos de master y msdb.

Proyecto 5: Recuperación Completa después de Corrupción del Archivo MDF (similar al 3, pero se enfatiza en corrupción).

Enunciado:

Se detectó corrupción física en el archivo principal de datos de la base GradosTítulos, impidiendo el acceso a las tablas relacionadas con expedientes de titulación. Recupere completamente la base utilizando la secuencia de respaldos disponibles.

```
-- Se asume que la base está corrupta y no se puede abrir. Se procede a
restaurar desde respaldos.
-- Se debe tener cuidado de no usar la base dañada para crear respaldos de log
(puede no ser posible).
-- La estrategia es restaurar el último full, luego diferenciales y logs.

-- 1. Verificar los respaldos disponibles (por ejemplo, listar archivos).
-- 2. Restaurar full con NORECOVERY.
RESTORE DATABASE GradosTítulos
FROM DISK = 'C:\Backups\GradosTítulos\Full\GradosTítulos_Full_20260627.bak'
WITH NORECOVERY, REPLACE;

-- 3. Restaurar diferencial.
RESTORE DATABASE GradosTítulos
FROM DISK = 'C:\Backups\GradosTítulos\Diff\GradosTítulos_Diff_20260628.bak'
WITH NORECOVERY;

-- 4. Aplicar todos los logs desde el último diferencial hasta el momento de la
corrupción (o hasta el último disponible).
-- Se listan los archivos de log en orden y se restauran secuencialmente con
NORECOVERY.
-- Ejemplo:
RESTORE LOG GradosTítulos
FROM DISK =
'C:\Backups\GradosTítulos\Log\GradosTítulos_Log_20260628_100000.trn'
WITH NORECOVERY;

RESTORE LOG GradosTítulos
FROM DISK =
'C:\Backups\GradosTítulos\Log\GradosTítulos_Log_20260628_101500.trn'
WITH NORECOVERY;

-- ...

-- 5. Finalizar con RECOVERY.
RESTORE DATABASE GradosTítulos WITH RECOVERY;

-- 6. Ejecutar DBCC CHECKDB para verificar integridad.
DBCC CHECKDB('GradosTítulos') WITH NO_INFOMSGS;
```

Justificación técnica

Cuando el archivo MDF está corrupto, se debe restaurar desde los respaldos, ya que no se puede reparar in situ. Se aplican full, diff y logs para obtener la información más reciente posible.

Buenas prácticas

- Utilizar RESTORE VERIFYONLY antes de restaurar.
- Mantener respaldos regulares y verificar su integridad.
- Considerar el uso de WITH CHECKSUM al hacer respaldos.

Proyecto 6: Restauración de la Base de Datos en un Servidor de Contingencia

Enunciado:

La Universidad requiere habilitar un centro alternativo de recuperación para continuar los procesos de grados y títulos. Restaure la base en una instancia secundaria.

```
-- Similar al Proyecto 2, pero con un enfoque de contingencia: se debe
restaurar la base en el servidor secundario
-- y posiblemente mantenerla actualizada (por ejemplo, con log shipping). Aquí
solo se muestra la restauración inicial.

-- 1. Copiar los archivos de respaldo desde el servidor principal al secundario.
-- 2. Restaurar la base con diferentes rutas de archivos (ajustar según la
estructura del servidor secundario).
RESTORE DATABASE GradosTitulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627.bak
,
WITH
    MOVE 'GradosTitulos' TO 'F:\Data\GradosTitulos.mdf',
    MOVE 'GradosTitulos_log' TO 'G:\Logs\GradosTitulos.ldf',
    NORECOVERY,
    REPLACE;

-- 3. Restaurar diferencial y logs (si se desea tener el estado más reciente).
RESTORE DATABASE GradosTitulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTitulos\Diff\GradosTitulos_Diff_20260628.bak
,
WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK =
'\\ServidorPrincipal\Backups\GradosTitulos\Log\GradosTitulos_Log_20260628_10000
0.trn'
WITH NORECOVERY;

-- ... (todos los logs)

RESTORE DATABASE GradosTitulos WITH RECOVERY;

-- 4. Opcional: configurar log shipping para mantener sincronizada la base
secundaria.
-- (No se incluye en este script por simplicidad)
```

Justificación técnica

Permite poner en marcha un entorno de respaldo o pruebas. Se utilizan MOVE para reubicar archivos físicos según la estructura del nuevo servidor.

Buenas prácticas

- Verificar que la versión de SQL Server sea compatible.
- Asegurar que las rutas de destino existan y tengan espacio suficiente.
- Probar la restauración periódicamente.

TEMA 3: Mantenimiento y Verificación de Copias de Seguridad

A continuación se desarrollan los seis proyectos relacionados con la verificación, inventario, detección de respaldos obsoletos y automatización de validaciones para la base de datos GradosTítulos.

Proyecto 1: Verificación de Integridad de Backups

Enunciado:

Verifique que todos los respaldos generados para GradosTítulos sean recuperables.

```
-- =====
-- PROYECTO 1: Verificación de integridad de backups
-- =====
-- Crear una tabla para almacenar los resultados de verificación
USE GradosTítulos;
GO

CREATE TABLE Backup.VerificacionIntegridad (
    IdVerificacion INT IDENTITY(1,1) PRIMARY KEY,
    FechaVerificacion DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    TipoBackup VARCHAR(20), -- FULL, DIFF, LOG
    RutaArchivo NVARCHAR(500),
    Resultado VARCHAR(20), -- EXITOSO, FALLIDO
    MensajeError NVARCHAR(MAX) NULL
);
GO

-- Procedimiento que verifica todos los backups de GradosTítulos en una carpeta
CREATE OR ALTER PROCEDURE Backup.SP_VerificarIntegridadBackups
    @BackupFolder NVARCHAR(500) = 'C:\Backups\GradosTítulos\'
AS
BEGIN
    SET NOCOUNT ON;

    -- Obtener lista de archivos .bak y .trn mediante xp_cmdshell (requiere
    habilitar)
    -- Alternativa: usar tabla msdb.dbo.backupset para obtener rutas de backups
    recientes.
    -- Usaremos la tabla de msdb para obtener los backups registrados.

    DECLARE @backupSet TABLE (
        backup_set_id INT,
        type CHAR(1),
        physical_device_name NVARCHAR(500),
        backup_start_date DATETIME
    );

    INSERT INTO @backupSet
    SELECT
        b.backup_set_id,
        b.type,
        m.physical_device_name,
        b.backup_start_date
    FROM msdb.dbo.backupset b
    JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
    WHERE b.database_name = 'GradosTítulos'
```

```

        AND b.backup_start_date >= DATEADD(day, -30, GETDATE()) -- últimos 30
días
ORDER BY b.backup_start_date DESC;

DECLARE @device NVARCHAR(500);
DECLARE @type CHAR(1);
DECLARE @resultado VARCHAR(20);
DECLARE @mensaje NVARCHAR(MAX);
DECLARE @sql NVARCHAR(MAX);

DECLARE cur CURSOR FOR
    SELECT physical_device_name, type FROM @backupSet;

OPEN cur;
FETCH NEXT FROM cur INTO @device, @type;
WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        -- Verificar integridad con RESTORE VERIFYONLY
        SET @sql = 'RESTORE VERIFYONLY FROM DISK = ''' + @device + '''';
        EXEC sp_executesql @sql;
        SET @resultado = 'EXITOSO';
        SET @mensaje = NULL;
    END TRY
    BEGIN CATCH
        SET @resultado = 'FALLIDO';
        SET @mensaje = ERROR_MESSAGE();
    END CATCH

    -- Registrar resultado
    INSERT INTO Backup.VerificacionIntegridad (
        FechaVerificacion, TipoBackup, RutaArchivo, Resultado, MensajeError
    )
    VALUES (
        SYSDATETIME(),
        CASE @type WHEN 'D' THEN 'FULL' WHEN 'I' THEN 'DIFF' WHEN 'L' THEN
'LOG' ELSE 'OTRO' END,
        @device,
        @resultado,
        @mensaje
    );

    FETCH NEXT FROM cur INTO @device, @type;
END;
CLOSE cur;
DEALLOCATE cur;

-- Resumen de resultados
SELECT Resultado, COUNT(*) AS Cantidad
FROM Backup.VerificacionIntegridad
WHERE FechaVerificacion = (SELECT MAX(FechaVerificacion) FROM
Backup.VerificacionIntegridad)
GROUP BY Resultado;
END;
GO

-- Ejecutar verificación
EXEC Backup.SP_VerificarIntegridadBackups;

```

Justificación técnica

RESTORE VERIFYONLY valida la estructura y checksums de los archivos de backup, asegurando que sean legibles y no estén corruptos. Permite detectar fallos antes de necesitar una restauración real.

Buenas prácticas

- Ejecutar verificación inmediatamente después de cada backup.
- Almacenar resultados para auditoría.
- No confiar en un backup sin verificar.

Proyecto 2: Inventario Histórico de Copias de Seguridad

Enunciado:

Construya un reporte que muestre todos los respaldos realizados durante el último mes.

```
-- =====
-- PROYECTO 2: Inventario histórico de backups (último mes)
-- =====
CREATE OR ALTER VIEW Backup.V_InventarioBackups
AS
SELECT
    b.database_name,
    b.backup_start_date,
    b.backup_finish_date,
    DATEDIFF(SECOND, b.backup_start_date, b.backup_finish_date) AS
DuracionSegundos,
    b.type AS TipoBackup,
    CASE b.type
        WHEN 'D' THEN 'COMPLETO'
        WHEN 'I' THEN 'DIFERENCIAL'
        WHEN 'L' THEN 'LOG'
        ELSE 'OTRO'
    END AS TipoDescripcion,
    b.backup_size / 1024.0 / 1024.0 AS TamanoMB,
    b.compressed_backup_size / 1024.0 / 1024.0 AS TamanoComprimidoMB,
    b.is_compressed,
    b.is_copy_only,
    m.physical_device_name AS RutaArchivo,
    b.user_name,
    b.server_name
FROM msdb.dbo.backupset b
JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
WHERE b.database_name = 'GradosTitulos'
    AND b.backup_start_date >= DATEADD(MONTH, -1, GETDATE())
    AND b.backup_start_date <= GETDATE();
GO

-- Consultar el inventario
SELECT * FROM Backup.V_InventarioBackups
ORDER BY backup_start_date DESC;
```

Justificación técnica

La tabla msdb.dbo.backupset almacena metadatos de todos los backups realizados. Consultarla permite obtener un reporte completo y detallado de los respaldos.

Buenas prácticas

- Usar vistas para simplificar consultas frecuentes.
- Incluir columnas como tamaño, duración y compresión.
- Automatizar la generación del reporte mensual.

Proyecto 3: Detección de Respaldos Obsoletos

Enunciado:

Identifique copias de seguridad que excedan la política institucional de retención (ej. retención de 30 días).

```
-- =====
-- PROYECTO 3: Detección de respaldos obsoletos (>30 días)
-- =====
CREATE OR ALTER VIEW Backup.V_BackupsObsoletos
AS
SELECT
    b.database_name,
    b.backup_start_date,
    b.type AS TipoBackup,
    CASE b.type
        WHEN 'D' THEN 'COMPLETO'
        WHEN 'I' THEN 'DIFERENCIAL'
        WHEN 'L' THEN 'LOG'
        ELSE 'OTRO'
    END AS TipoDescripcion,
    b.backup_size / 1024.0 / 1024.0 AS TamanoMB,
    m.physical_device_name AS RutaArchivo,
    DATEDIFF(DAY, b.backup_start_date, GETDATE()) AS DiasAntiguedad
FROM msdb.dbo.backupset b
JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
WHERE b.database_name = 'GradosTitulos'
    AND b.backup_start_date < DATEADD(DAY, -30, GETDATE()) -- Política:
    retención 30 días
    AND b.backup_start_date IS NOT NULL;
GO

-- Reporte de obsoletos
SELECT * FROM Backup.V_BackupsObsoletos
ORDER BY backup_start_date;
}
```

Justificación técnica

Una política de retención define cuánto tiempo conservar los backups. Identificar los que excedan el límite permite liberar espacio y cumplir con normativas.

Buenas prácticas

- Establecer retención según RPO/RTO y requisitos legales.
- Eliminar automáticamente archivos obsoletos con scripts (ej. PowerShell) o trabajos de mantenimiento.
- Documentar la política.

Proyecto 4: Sistema Automatizado de Verificación (Diario)

Enunciado:

Diseña una solución que valide diariamente la integridad de todos los respaldos de GradosTítulos.

```
-- =====
-- PROYECTO 4: Sistema automatizado de verificación diaria
-- =====
-- Procedimiento que verifica los backups del día anterior
CREATE OR ALTER PROCEDURE Backup.SP_VerificarBackupsDiarios
AS
BEGIN
    SET NOCOUNT ON;

    -- Verificar backups del día anterior
    DECLARE @fechaInicio DATETIME = DATEADD(DAY, -1, GETDATE());
    DECLARE @fechaFin DATETIME = GETDATE();

    DECLARE @backupSet TABLE (
        physical_device_name NVARCHAR(500),
        type CHAR(1)
    );

    INSERT INTO @backupSet
    SELECT DISTINCT
        m.physical_device_name,
        b.type
    FROM msdb.dbo.backupset b
    JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
    WHERE b.database_name = 'GradosTítulos'
        AND b.backup_start_date BETWEEN @fechaInicio AND @fechaFin;

    DECLARE @device NVARCHAR(500), @type CHAR(1), @resultado VARCHAR(20),
    @mensaje NVARCHAR(MAX), @sql NVARCHAR(MAX);

    DECLARE cur CURSOR FOR SELECT physical_device_name, type FROM @backupSet;
    OPEN cur;
    FETCH NEXT FROM cur INTO @device, @type;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        BEGIN TRY
            SET @sql = 'RESTORE VERIFYONLY FROM DISK = ''' + @device + ''';
            EXEC sp_executesql @sql;
            SET @resultado = 'EXITOSO';
            SET @mensaje = NULL;
        END TRY
        BEGIN CATCH
            SET @resultado = 'FALLIDO';
            SET @mensaje = ERROR_MESSAGE();
        END CATCH

        INSERT INTO Backup.VerificacionIntegridad (FechaVerificacion,
        TipoBackup, RutaArchivo, Resultado, MensajeError)
        VALUES (SYSDATETIME(), CASE @type WHEN 'D' THEN 'FULL' WHEN 'I' THEN
        'DIFF' WHEN 'L' THEN 'LOG' ELSE 'OTRO' END,
            @device, @resultado, @mensaje);

        FETCH NEXT FROM cur INTO @device, @type;
    END;
    CLOSE cur;
```

```

DEALLOCATE cur;

-- Enviar alerta si hay fallos
IF EXISTS (SELECT 1 FROM Backup.VerificacionIntegridad WHERE
FechaVerificacion >= @fechaInicio AND Resultado = 'FALLIDO')
BEGIN
    -- Enviar correo o registrar evento (ejemplo con sp_send_dbmail)
    EXEC msdb.dbo.sp_send_dbmail
        @recipients = 'dba@universidad.edu',
        @subject = 'ALERTA: Fallo en verificación de backups de
GradosTitulos',
        @body = 'Se detectaron backups fallidos. Revisar tabla
Backup.VerificacionIntegridad.';
END
END;
GO

-- Crear un job en SQL Agent que ejecute este procedimiento diariamente a las
06:00
-- (Se puede crear con el asistente o con script)

```

Justificación técnica

Automatizar la validación reduce la carga manual y garantiza que se detecten fallos a tiempo. El procedimiento verifica los backups del día anterior y envía alertas ante errores.

Buenas prácticas

- Programar en horas de bajo impacto.
- Configurar alertas por correo o en la herramienta de monitoreo.
- Revisar los logs de verificación periódicamente.

Proyecto 5: Sistema Automatizado de Validación de Backups

Enunciado:

Diseñe un mecanismo que valide diariamente todos los respaldos de la base GradosTítulos. Nota: Este proyecto es muy similar al anterior. Se puede reutilizar el procedimiento SP_VerificarBackupsDiarios, pero se puede añadir validación adicional (ej. comprobar que el tamaño del backup no sea cero, que el backup se haya completado correctamente, etc.).

```
-- =====
-- PROYECTO 5: Validación diaria de backups (ampliada)
-- =====
CREATE OR ALTER PROCEDURE Backup.SP_ValidarBackupsDiarios
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @fechaInicio DATETIME = DATEADD(DAY, -1, GETDATE());
    DECLARE @fechaFin DATETIME = GETDATE();

    -- Validar que existan backups de cada tipo en el último día
    DECLARE @fullCount INT, @diffCount INT, @logCount INT;

    SELECT @fullCount = COUNT(*) FROM msdb.dbo.backupset
    WHERE database_name = 'GradosTítulos' AND type = 'D'
    AND backup_start_date BETWEEN @fechaInicio AND @fechaFin;

    SELECT @diffCount = COUNT(*) FROM msdb.dbo.backupset
    WHERE database_name = 'GradosTítulos' AND type = 'I'
    AND backup_start_date BETWEEN @fechaInicio AND @fechaFin;

    SELECT @logCount = COUNT(*) FROM msdb.dbo.backupset
    WHERE database_name = 'GradosTítulos' AND type = 'L'
    AND backup_start_date BETWEEN @fechaInicio AND @fechaFin;

    -- Si falta algún tipo, registrar alerta
    IF @fullCount = 0 OR @diffCount = 0 OR @logCount = 0
    BEGIN
        -- Insertar en tabla de alertas
        INSERT INTO Backup.AlertasBackup (Fecha, Mensaje)
        VALUES (SYSDATETIME(),
            CONCAT('Faltan backups en el último día: Full=', @fullCount, ',
Diff=', @diffCount, ', Log=', @logCount));
    END

    -- Ejecutar verificación de integridad (reutilizar procedimiento anterior)
    EXEC Backup.SP_VerificarBackupsDiarios;
END;
GO
```

Justificación técnica

Además de integridad, se comprueba que existan backups de todos los tipos (Full, Diff, Log) en el período diario, detectando ausencias que podrían comprometer la recuperación.

Buenas prácticas

- Combinar verificación de integridad con validación de cobertura.
- Registrar todas las alertas en una tabla centralizada.
- Establecer acciones correctivas automáticas cuando sea posible.

Proyecto 6: Auditoría Integral del Historial de Copias de Seguridad

Enunciado:

La Dirección de TI solicita un reporte detallado de los respaldos ejecutados sobre GradosTítulos durante los últimos 90 días.

```
-- =====
-- PROYECTO 6: Auditoría integral de backups (90 días)
-- =====
CREATE OR ALTER VIEW Backup.V_AuditoriaBackups90Dias
AS
SELECT
    b.database_name,
    b.backup_start_date,
    b.backup_finish_date,
    DATEDIFF(SECOND, b.backup_start_date, b.backup_finish_date) AS
DuracionSegundos,
    CASE b.type
        WHEN 'D' THEN 'COMPLETO'
        WHEN 'I' THEN 'DIFERENCIAL'
        WHEN 'L' THEN 'LOG'
        ELSE 'OTRO'
    END AS TipoBackup,
    CAST(b.backup_size / 1048576.0 AS DECIMAL(10,2)) AS TamanoMB,
    CAST(b.compressed_backup_size / 1048576.0 AS DECIMAL(10,2)) AS
TamanoComprimidoMB,
    b.is_compressed,
    b.is_copy_only,
    b.is_damaged,
    b.has_backup_checksums,
    b.is_password_protected,
    m.physical_device_name AS RutaArchivo,
    b.user_name,
    b.server_name,
    b.recovery_model,
    b.first_lsn,
    b.last_lsn,
    b.checkpoint_lsn,
    b.database_backup_lsn,
    b.begins_log_chain,
    b.has_incomplete_metadata,
    b.is_snapshot
FROM msdb.dbo.backupset b
JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
WHERE b.database_name = 'GradosTítulos'
    AND b.backup_start_date >= DATEADD(DAY, -90, GETDATE())
    AND b.backup_start_date <= GETDATE();
GO
```



```

-- Reporte resumido por tipo de backup
CREATE OR ALTER VIEW Backup.V_ResumenBackups90Dias
AS
SELECT
    TipoBackup,
    COUNT(*) AS Cantidad,
    SUM(TamanoMB) AS TotalMB,
    AVG(DuracionSegundos) AS PromedioDuracionSeg,
    MIN(backup_start_date) AS PrimerBackup,
    MAX(backup_start_date) AS UltimoBackup
FROM Backup.V_AuditoriaBackups90Dias
GROUP BY TipoBackup;
GO

-- Ejecutar consultas
SELECT * FROM Backup.V_AuditoriaBackups90Dias ORDER BY backup_start_date DESC;
SELECT * FROM Backup.V_ResumenBackups90Dias;

```

Justificación técnica

Proporciona un reporte detallado con métricas de todos los backups, útil para auditorías externas, análisis de rendimiento y planificación de capacidad

Buenas prácticas

- Incluir campos como LSN, estado de compresión, checksums, etc.
- Programar la generación del reporte mensual para la dirección.
- Conservar el historial de auditoría durante al menos 1 año.

TEMA 5: PLANES DE MANTENIMIENTO Y POLITICAS DE RETENCION

Proyecto 1: Creación de un Plan de Mantenimiento Semanal

Enunciado:

Creación de un Plan de Mantenimiento Semanal.

```
-- =====
-- PROYECTO 1: Plan de mantenimiento semanal
-- =====
-- Se crea un procedimiento que ejecuta todas las tareas de mantenimiento:
-- 1. Respaldo completo (Full)
-- 2. Reorganización de índices (con fragmentación > 5%)
-- 3. Actualización de estadísticas
-- 4. Verificación de integridad de la base de datos (DBCC CHECKDB)

CREATE OR ALTER PROCEDURE Maintenance.SP_MantenimientoSemanal
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Backup completo (con compresión y checksum)
    DECLARE @backupPath NVARCHAR(500) = 'C:\Backups\GradosTitulos\Full\';
    DECLARE @fileName NVARCHAR(500) = @backupPath + 'GradosTitulos_Full_'
        + FORMAT(GETDATE(), 'yyyyMMdd_HHmss') + '.bak';
    EXEC xp_create_subdir @backupPath;
    BACKUP DATABASE GradosTitulos TO DISK = @fileName
    WITH COMPRESSION, INIT, CHECKSUM, STATS = 10;

    -- 2. Reorganizar índices (si fragmentación > 5% y < 30%, reorganizar; >
    30% reconstruir)
    -- Se usa sp_MSforeachtable, pero se recomienda un enfoque más controlado.
    DECLARE @sql NVARCHAR(MAX) = N'';
    SELECT @sql = @sql +
        'IF EXISTS (SELECT 1 FROM sys.dm_db_index_physical_stats(DB_ID(),
        OBJECT_ID('' + QUOTENAME(SCHEMA_NAME(schema_id)) + '.' + QUOTENAME(name) +
        '''), NULL, NULL, ''LIMITED'') WHERE avg_fragmentation_in_percent > 5)
        BEGIN
            ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(schema_id)) + '.' +
            QUOTENAME(name) + ' REORGANIZE;
        END'
    FROM sys.tables WHERE is_ms_shipped = 0;
    EXEC sp_executesql @sql;

    -- 3. Actualizar estadísticas en todas las tablas
    EXEC sp_updatestats;

    -- 4. Verificar integridad de la base de datos
    DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;

    -- Registrar la ejecución
    INSERT INTO Maintenance.HistorialMantenimiento (FechaEjecucion, Tarea,
    Estado)
    VALUES (SYSDATETIME(), 'MantenimientoSemanal', 'EXITOSO');
END;
GO

-- Crear tabla de historial (si no existe)
CREATE TABLE Maintenance.HistorialMantenimiento (
```

```

IdHistorial INT IDENTITY(1,1) PRIMARY KEY,
FechaEjecucion DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
Tarea VARCHAR(100),
Estado VARCHAR(20)
);
GO
-- Crear un trabajo en SQL Agent para ejecutar este procedimiento cada domingo
a las 02:00
-- (Se puede hacer desde SSMS o con script, pero se omite por brevedad)

```

Justificación técnica

Un mantenimiento semanal garantiza que la base de datos esté respaldada, los índices optimizados y las estadísticas actualizadas, lo que mejora el rendimiento y la capacidad de recuperación.

Buenas prácticas

- Programar en horario de bajo uso.
- Monitorear la duración de las tareas.
- Validar los resultados con el historial.

Proyecto 2: Política de Retención de 90 Días

Enunciado:

Implemente una política institucional para conservar respaldos durante 90 días.

```

-- =====
-- PROYECTO 2: Política de retención de 90 días
-- =====
-- Procedimiento que elimina archivos de backup con más de 90 días de
antigüedad
-- Usa la tabla msdb.dbo.backupset para identificar los archivos obsoletos,
-- y luego los elimina del sistema de archivos (requiere xp_cmdshell
habilitado).

CREATE OR ALTER PROCEDURE Maintenance.SP_EliminarBackupsAntiguos
    @RetencionDias INT = 90
AS
BEGIN
    SET NOCOUNT ON;

    -- Obtener rutas de archivos de backup con más de @RetencionDias días
    DECLARE @archivos TABLE (Ruta NVARCHAR(500), Fecha DATETIME);
    INSERT INTO @archivos
    SELECT DISTINCT m.physical_device_name, MAX(b.backup_start_date)
    FROM msdb.dbo.backupset b
    JOIN msdb.dbo.backupmediafamily m ON b.media_set_id = m.media_set_id
    WHERE b.database_name = 'GradosTitulos'
    AND b.backup_start_date < DATEADD(DAY, -@RetencionDias, GETDATE())
    GROUP BY m.physical_device_name;

    DECLARE @ruta NVARCHAR(500);
    DECLARE @cmd NVARCHAR(4000);

    DECLARE cur CURSOR FOR SELECT Ruta FROM @archivos;
    OPEN cur;
    FETCH NEXT FROM cur INTO @ruta;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Eliminar archivo físicamente (xp_cmdshell debe estar habilitado)
    
```

```

        SET @cmd = 'del "' + @ruta + '"';
        EXEC xp_cmdshell @cmd;
        FETCH NEXT FROM cur INTO @ruta;
    END;
    CLOSE cur;
    DEALLOCATE cur;

    -- Registrar en historial
    INSERT INTO Maintenance.HistorialMantenimiento (Tarea, Estado)
    VALUES ('EliminacionBackups > ' + CAST(@RetencionDias AS VARCHAR) + ' dias',
    'EXITOSO');
END;
GO

-- Programar ejecución diaria (SQL Agent job)

```

Justificación técnica

Eliminar backups antiguos libera espacio y cumple con políticas de retención estándar. Usar msdb.dbo.backupset para identificar archivos obsoletos garantiza consistencia.

Buenas prácticas

- Utilizar xp_cmdshell con precaución o usar scripts externos.
- Registrar la eliminación para auditoría.
- Mantener un catálogo de backups archivados.

Proyecto 3: Plan de Mantenimiento Integral

Enunciado:

Diseña un plan que incluya: Backups, Reorganización de índices, Actualización de estadísticas, Verificación de integridad.

```

-- =====
-- PROYECTO 3: Plan de mantenimiento integral (diario + semanal)
-- =====
-- Este plan combina tareas diarias (log backup, actualización estadísticas
-- ligera)
-- y semanales (full backup, reorganización, CHECKDB).

-- 1. Tarea diaria (ejecutar cada día a las 23:00)
CREATE OR ALTER PROCEDURE Maintenance.SP_MantenimientoDiario
AS
BEGIN
    -- Backup del log de transacciones (cada 30 min en otro job)
    -- Aquí solo actualizamos estadísticas y verificamos integridad rápida
    EXEC sp_updatestats; -- Actualización de estadísticas con muestreo
    predeterminado

    -- Verificación de integridad de solo lectura (no bloqueante)
    DBCC CHECKDB('GradosTitulos') WITH PHYSICAL_ONLY, NO_INFOMSGS;

    -- Registrar
    INSERT INTO Maintenance.HistorialMantenimiento (Tarea, Estado)
    VALUES ('MantenimientoDiario', 'EXITOSO');
END;
GO

-- 2. Tarea semanal (domingo) - ya creada en el Proyecto 1, pero la ampliamos
-- Para integrar, podemos modificar el procedimiento semanal para incluir:

```

```

-- - Full backup
-- - Reorganización de índices (más exhaustiva)
-- - Actualización de estadísticas con escaneo completo
-- - CHECKDB completo

ALTER PROCEDURE Maintenance.SP_MantenimientoSemanal
AS
BEGIN
    -- (código del proyecto 1, pero con mejoras)
    -- Se puede añadir también:
    -- - Reconstrucción de índices con fragmentación > 30%
    -- - Actualización de estadísticas con FULLSCAN
    -- - CHECKDB completo
    -- -- (ya implementado)
END;
GO

-- 3. Crear jobs en SQL Agent para ejecutar diario y semanal
-- Se omite el script de creación de jobs por brevedad.

```

Justificación técnica

Combinar backups, índices, estadísticas y verificación de integridad en un solo plan asegura un estado saludable de la base de datos. La verificación temprana detecta corrupción.

Buenas prácticas

- Separar tareas diarias (ligeras) de semanales (pesadas).
- Usar PHYSICAL_ONLY en CHECKDB para reducir impacto.
- Probar la restauración de backups regularmente.

Proyecto 4: Gestión Corporativa de Retención (5 años)

Enunciado:

Diseña una política institucional de respaldo y retención para cinco años de operación.

```

-- =====
-- PROYECTO 4: Política de retención de 5 años
-- =====
-- Estrategia:
-- - Respaldos completos semanales se conservan por 5 años (archivados en cinta o almacenamiento frío).
-- - Respaldos diferenciales y logs se conservan por 30 días (para recuperación rápida).
-- - Se implementa una tabla de catálogo para rastrear respaldos archivados.

-- 1. Crear tabla de catálogo de respaldos archivados
CREATE TABLE Backup.CatalogoBackups (
    IdBackup INT IDENTITY(1,1) PRIMARY KEY,
    FechaBackup DATETIME2 NOT NULL,
    TipoBackup VARCHAR(10), -- FULL, DIFF, LOG
    RutaArchivo NVARCHAR(500),
    FechaArchivo DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    UbicacionArchivo VARCHAR(50) -- 'DISCO_LOCAL', 'CINTA', 'AZURE'
);

-- 2. Procedimiento para mover backups a almacenamiento de largo plazo (> 90 días)
CREATE OR ALTER PROCEDURE Backup.SP_ArchivarBackupsAntiguos
AS

```

```

BEGIN
    -- Seleccionar backups con más de 90 días y que aún no estén archivados
    -- Mover físicamente los archivos a otra ubicación (ej. unidad de red,
Azure Blob)
    -- Registrar en la tabla CatalogoBackups
    -- Luego eliminar de la ubicación original (ya se hizo en el proyecto 2)
    -- Este procedimiento se ejecutaría mensualmente.
    -- Se omite implementación detallada por complejidad.
END;
GO

-- 3. Política de retención a 5 años:
-- - Los backups completos semanales se conservan indefinidamente (o 5 años)
en almacenamiento frío.
-- - Se recomienda usar Azure Backup o similar para retención a largo plazo.
-- - Configurar en SQL Server Maintenance Plans (GUI) o mediante scripts.

```

Justificación técnica

La retención a largo plazo es necesaria por requisitos legales y de auditoría. Se requiere almacenamiento fuera de línea (cintas, Azure Archive).

Buenas prácticas

- Definir políticas claras de archivado.
- Mantener un catálogo centralizado.
- Automatizar la migración a almacenamiento frío.

Proyecto 5: Plan Corporativo de Mantenimiento

Enunciado:

Diseñe un plan de mantenimiento que incluya: Respaldo completo, Reorganización de índices, Actualización de estadísticas, Validación de integridad.

```

-- =====
-- PROYECTO 5: Plan Corporativo de Mantenimiento
-- =====
-- Similar al proyecto 3, pero con un enfoque más robusto:
-- - Uso de Ola Hallengren scripts (recomendado) o creación propia.
-- - Aquí se muestra un procedimiento unificado que ejecuta todas las tareas
--   con parámetros configurables.

CREATE OR ALTER PROCEDURE Maintenance.SP_MantenimientoCorporativo
    @Tipo VARCHAR(10) = 'COMPLETO' -- 'COMPLETO' o 'LIGERO'
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @fechaInicio DATETIME = SYSDATETIME();
    DECLARE @estado VARCHAR(20) = 'EXITOSO';

    BEGIN TRY
        -- 1. Respaldo completo (si es COMPLETO)
        IF @Tipo = 'COMPLETO'
        BEGIN
            DECLARE @path NVARCHAR(500) = 'C:\Backups\GradosTitulos\Full\';
            DECLARE @file NVARCHAR(500) = @path + 'GradosTitulos_Full_' +
FORMAT(GETDATE(), 'yyyyMMdd_HHmmss') + '.bak';
            EXEC xp_create_subdir @path;
            BACKUP DATABASE GradosTitulos TO DISK = @file WITH COMPRESSION,
INIT, CHECKSUM;
        END
    END TRY
    BEGIN CATCH
        -- Manejo de errores
    END CATCH
END

```

```

-- 2. Reorganización/Reconstrucción de índices
DECLARE @sql NVARCHAR(MAX) = '';
SELECT @sql = @sql +
    'ALTER INDEX ALL ON ' + QUOTENAME(SCHEMA_NAME(schema_id)) + '.' +
    QUOTENAME(name) + ' REORGANIZE;'
FROM sys.tables WHERE is_ms_shipped = 0;
EXEC sp_executesql @sql;

-- 3. Actualización de estadísticas
EXEC sp_updatestats;

-- 4. Verificación de integridad
DBCC CHECKDB('GradosTitulos') WITH NO_INFOMSGS;

END TRY
BEGIN CATCH
    SET @estado = 'FALLIDO: ' + ERROR_MESSAGE();
END CATCH

-- Registrar
INSERT INTO Maintenance.HistorialMantenimiento (FechaEjecucion, Tarea,
Estado)
VALUES (SYSDATETIME(), 'MantenimientoCorporativo_' + @Tipo, @estado);
END;
GO

```

Justificación técnica

Un plan parametrizable permite adaptarse a diferentes entornos (producción, pruebas) y facilita la estandarización en toda la organización.

Buenas prácticas

- Usar scripts modulares y reutilizables.
- Incluir manejo de errores y registro.
- Documentar los parámetros y su uso.

Proyecto 6: Política de Retención Automatizada (180 días)

Enunciado:

Diseña una política institucional que elimine automáticamente respaldos con más de 180 días de antigüedad.

```

-- =====
-- PROYECTO 6: Política de retención de 180 días
-- =====
-- Se reutiliza el procedimiento del proyecto 2, pero con retención de 180 días.
-- Se puede crear un job diario que llame al procedimiento con 180.

CREATE OR ALTER PROCEDURE Maintenance.SP_EliminarBackups180Dias
AS
BEGIN
    EXEC Maintenance.SP_EliminarBackupsAntiguos @RetencionDias = 180;
END;
GO

-- Adicionalmente, se puede crear un procedimiento que también limpie archivos
-- de la carpeta de logs y diferenciales con la misma política.
CREATE OR ALTER PROCEDURE Maintenance.SP_LimpiarArchivosBackups

```

```

AS
BEGIN
    -- Usar xp_cmdshell para eliminar archivos por antigüedad en las carpetas
    -- Ejemplo: eliminar archivos .bak, .trn con más de 180 días en
    C:\Backups\GradosTitulos\
    DECLARE @cmd NVARCHAR(4000) =
        'forfiles /p "C:\Backups\GradosTitulos" /s /m *.bak /d -180 /c "cmd /c
del @path"';
    EXEC xp_cmdshell @cmd;

    SET @cmd = 'forfiles /p "C:\Backups\GradosTitulos" /s /m *.trn /d -180 /c
"cmd /c del @path"';
    EXEC xp_cmdshell @cmd;

    -- Registrar
    INSERT INTO Maintenance.HistorialMantenimiento (Tarea, Estado)
    VALUES ('LimpiarBackups180Dias', 'EXITOSO');
END;
GO

```

Justificación técnica

Extender la retención a 180 días puede ser requerido por normativas internas más estrictas. La automatización evita la acumulación de archivos.

Buenas prácticas

- Ajustar la periodicidad de limpieza según el crecimiento.
- Combinar con alertas de espacio en disco.
- Validar que los backups más recientes no se eliminen.

TEMA 6: Restauración Punto en el Tiempo

A continuación se desarrollan los seis proyectos relacionados con la recuperación de la base de datos GradosTitulos a un momento específico (Point-in-Time Recovery). Se asume que la base de datos está en modo FULL RECOVERY y que se dispone de una cadena de respaldos completa (Full + Diferenciales + Logs). Los scripts utilizan RESTORE con STOPAT y manejo de NORECOVERY/RECOVERY.

Proyecto 1: Recuperación Antes de una Eliminación Accidental (10:15 a.m.)

Enunciado:

Un operador eliminó registros de expedientes a las 10:15 a.m. Restaure la base exactamente a las 10:14 a.m.

```
-- =====
-- PROYECTO 1: Recuperar a las 10:14:00
-- =====
-- Se asume que los respaldos están en las rutas estándar.
-- Se usa el procedimiento genérico con la hora objetivo.

EXEC Restore.SP_RestaurarPuntoEnTiempo
    @TargetDateTime = '2026-06-28 10:14:00.000',
    @FullBackupPath =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak', --
ejemplo
    @DiffBackupFolder = 'C:\Backups\GradosTitulos\Diff\', -- si existe
    @LogBackupFolder = 'C:\Backups\GradosTitulos\Log\';

-- Verificar que los datos estén recuperados
SELECT COUNT(*) FROM Tramite.Tramite WHERE FechaInicio >= '2026-06-28'; --
comprobación
```

Justificación técnica

La restauración a un punto específico permite deshacer eliminaciones sin perder los cambios posteriores válidos. Se usa STOPAT para detener la aplicación del log en el instante previo a la eliminación.

Buenas prácticas

- Conocer la hora exacta del incidente
- Probar la restauración en un entorno de pruebas
- Documentar el procedimiento.

Proyecto 2: Recuperación de Cambios Incorrectos

Enunciado:

Un proceso masivo actualizó incorrectamente estados de trámites. Se requiere restaurar al momento anterior a la actualización (por ejemplo, 09:30 a.m.)

```
-- =====
-- PROYECTO 2: Recuperar a las 09:30:00 (antes de la actualización masiva)
-- =====

EXEC Restore.SP_RestaurarPuntoEnTiempo
    @TargetDateTime = '2026-06-28 09:30:00.000',
    @FullBackupPath =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak',
    @DiffBackupFolder = 'C:\Backups\GradosTitulos\Diff\',
    @LogBackupFolder = 'C:\Backups\GradosTitulos\Log\';
```

```
-- Después de la restauración, verificar los estados
SELECT IdEstadoTramite, COUNT(*) FROM Tramite.Tramite GROUP BY IdEstadoTramite;
```

Justificación técnica

Similar al anterior, pero para corregir actualizaciones masivas. Es crucial identificar la hora de inicio del proceso erróneo.

Buenas prácticas

- Usar transacciones y pruebas en desarrollo.
- Realizar backups de log frecuentes para reducir el RPO.

Proyecto 3: Recuperación Selectiva de Incidente Crítico

Enunciado:

Un error en producción afectó las tablas Tramite y Evaluacion. Diseñe una recuperación punto en el tiempo minimizando la indisponibilidad. Estrategia: Restaurar la base en un servidor alternativo (o con nombre diferente) hasta el punto anterior al error. Extraer los datos afectados y volcarlos a la base original, o conmutar a la base restaurada.

```
-- =====
-- PROYECTO 3: Recuperación selectiva con mínima indisponibilidad
-- =====
-- 1. Restaurar en una base nueva (GradosTitulos_Recuperada) en el mismo
servidor
-- para no interrumpir la producción mientras se recupera.

DECLARE @restoreSql NVARCHAR(MAX);
SET @restoreSql = '
RESTORE DATABASE GradosTitulos_Recuperada
FROM DISK =
''C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak''
WITH
    MOVE ''GradosTitulos'' TO ''D:\Data\GradosTitulos_Recuperada.mdf'',
    MOVE ''GradosTitulos_log'' TO ''D:\Logs\GradosTitulos_Recuperada_log.ldf'',
    NORECOVERY, REPLACE;
';
EXEC sp_executesql @restoreSql;

-- 2. Aplicar diferencial y logs hasta el punto deseado (ejemplo 10:00)
-- (se puede usar el procedimiento genérico adaptado para la nueva base)
-- Se omite para no repetir, pero se haría con RESTORE LOG ... STOPAT = '2026-
06-28 10:00:00'

-- 3. Una vez restaurada, se puede intercambiar el nombre de la base o exportar
los datos.
-- Por ejemplo, usar scripts de sincronización.

-- 4. Finalmente, poner la base en producción.
```

Justificación técnica

Restaurar en un servidor alternativo o con nombre diferente permite mantener la producción activa mientras se recupera, luego se sincronizan solo los datos afectados.

Buenas prácticas

- Tener un servidor de contingencia listo
- Usar siempre NORECOVERY para poder aplicar logs adicionales.
- Planificar la ventana de conmutación.

Proyecto 4: Simulación de Recuperación Forense Institucional

Enunciado: Se detectó una modificación maliciosa realizada durante una ventana específica de tiempo. Realice una recuperación forense que permita restaurar la base GradosTítulos exactamente al momento anterior al incidente y documente todo el procedimiento.

```
-- =====
-- PROYECTO 4: Recuperación forense
-- =====
-- Supongamos que la modificación ocurrió entre las 14:00 y las 14:30 del
28/06/2026.
-- Restauramos al punto exacto anterior, p. ej. 13:59:59.

EXEC Restore.SP_RestaurarPuntoEnTiempo
    @TargetDateTime = '2026-06-28 13:59:59.999',
    @FullBackupPath =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak',
    @DiffBackupFolder = 'C:\Backups\GradosTitulos\Diff\',
    @LogBackupFolder = 'C:\Backups\GradosTitulos\Log\';

-- Documentación del procedimiento:
-- 1. Se identificó la ventana de tiempo sospechosa (14:00 - 14:30).
-- 2. Se seleccionó el backup completo más reciente (27/06).
-- 3. Se aplicaron diferenciales (si los hubiera) y logs hasta 13:59:59.999.
-- 4. Se verificó la integridad de los datos restaurados.
-- 5. Se realizó un análisis de cambios (comparativa de tablas) para
identificar las filas modificadas.
-- 6. Se generó un reporte detallado de la recuperación.

-- Consulta para verificar que los datos estén correctos
SELECT * FROM Documento.Resolucion WHERE FechaEmision >= '2026-06-28'; --
Comparar con backup
```

Justificación técnica

La recuperación forense requiere precisión milimétrica y documentación exhaustiva para posibles investigaciones. Se restaura a un punto anterior y se comparan los datos para identificar el alcance del daño.

Buenas prácticas

- Mantener un registro de auditoría de cambios
- Usar STOPAT con la máxima precisión.
- Realizar un informe detallado de la restauración.

Proyecto 5: Recuperación Exacta de Expedientes Eliminados (11:45 a.m.)

Enunciado:

A las 11:45 a.m. se eliminaron accidentalmente registros de expedientes de titulación. Recupere la base exactamente a las 11:44:59 a.m.

```
-- =====
-- PROYECTO 5: Recuperar a las 11:44:59
-- =====
EXEC Restore.SP_RestaurarPuntoEnTiempo
    @TargetDateTime = '2026-06-28 11:44:59.000',
    @FullBackupPath =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak',
    @DiffBackupFolder = 'C:\Backups\GradosTitulos\Diff\',
    @LogBackupFolder = 'C:\Backups\GradosTitulos\Log\';

-- Verificar que los registros eliminados hayan vuelto
SELECT COUNT(*) FROM Tramite.Tramite WHERE FechaInicio = '2026-06-28';
```

Justificación técnica

La restauración a un punto específico permite deshacer eliminaciones sin perder los cambios posteriores válidos. Se usa STOPAT para detener la aplicación del log en el instante previo a la eliminación.

Buenas prácticas

- Conocer la hora exacta del incidente.
- Probar la restauración en un entorno de pruebas.
- Documentar el procedimiento.

Proyecto 6: Recuperación Forense de Modificación Maliciosa en Documento

Enunciado:

Se detectó una modificación no autorizada en las tablas del esquema Documento (resoluciones y diplomas). Realice una recuperación punto en el tiempo que permita restaurar el estado exacto de la base antes de la alteración.

```
-- =====
-- PROYECTO 6: Recuperación forense de Documento
-- =====
-- Se presume que la alteración ocurrió entre las 16:00 y 16:15.
-- Restauramos al instante anterior: 15:59:59.

EXEC Restore.SP_RestaurarPuntoEnTiempo
    @TargetDateTime = '2026-06-28 15:59:59.999',
    @FullBackupPath =
'C:\Backups\GradosTitulos\Full\GradosTitulos_Full_20260627_020000.bak',
    @DiffBackupFolder = 'C:\Backups\GradosTitulos\Diff\',
    @LogBackupFolder = 'C:\Backups\GradosTitulos\Log\';

-- Después de la restauración, se puede verificar la integridad:
-- Comparar el número de registros antes y después del incidente.
-- También se pueden restaurar solo esas tablas en una base paralela y extraer los datos.
```

Justificación técnica

La recuperación forense requiere precisión milimétrica y documentación exhaustiva para posibles investigaciones. Se restaura a un punto anterior y se comparan los datos para identificar el alcance del daño.

Buenas prácticas

- Mantener un registro de auditoría de cambios.
- Usar STOPAT con la máxima precisión.
- Realizar un informe detallado de la restauración